

Abolish the term "converting constructor"

Document number: P3542R0

Date: December 15, 2024

Audience: EWG, CWG

Reply to: Brian Bi (bbi5291@gmail.com)

Situation

The Standard defines the term *converting constructor* as “A constructor that is not explicit” ([class.conv.ctor]/1). This term is then used in four places in the Standard: once to redundantly point out that a non-explicit copy or move constructor counts as a converting constructor, twice to specify that non-explicit constructors are candidates for certain initializations, and once in a note.

The first use should give the reader pause. Why does the Standard point out that a non-explicit copy/move constructor is a converting constructor; shouldn't this fact be an entirely obvious consequence of the fact that “converting constructor” means any non-explicit constructor? Unfortunately, the choice of terminology seems to be at odds with the definition: a copy or move constructor initializes an object from another object of the same type (modulo cv-qualification) so it does not perform what would normally be thought of as a type conversion. It is therefore necessary to remind the reader that, in fact, copy and move constructors still qualify as *converting constructors* (assuming they are not explicit).

Equally counterintuitive is the fact that default constructors can be converting constructors (again, as long as they are not explicit), and so can constructors that require more than one parameter. Such constructors can be used to form an *implicit conversion sequence* from an empty *braced-init-list*, or a list with two or more elements, respectively, to the constructor's class. However, C++ does not have actual *implicit conversions* where the source is a *braced-init-list*, so such constructors do not participate in implicit conversions even though they are called converting constructors. (Instead of implicitly converting the list to the destination class type, we *list-initialize* the object; [dcl.init.general]/16.1.)

A further problem is that the C++ language provides mechanisms to perform both implicit and explicit conversions; for example, [expr.static.cast]/5 describes circumstances under which “an expression *E* can be explicitly converted to a type *T*” by the `static_cast` operator (and specifies the semantics of such conversions). Explicit constructors are excluded from being considered converting constructors, even though they can still participate in explicit conversions. The notations `T{}` and `T{x, y}` can be used to invoke constructors that accept zero and two arguments, respectively, for an explicit conversion ([expr.type.conv]/2). In this context, converting constructors described in the previous paragraph can participate, but so can explicit constructors.

So, the use of the term *converting constructor* is problematic even within the Standard itself. The more significant problem, however, is that outside the Committee, people are using the term with a meaning different from the one in the Standard. Frequently, C++ programmers think that a converting constructor must accept a single argument or that it must have a parameter type that sufficiently differs from that of the constructor's class (*i.e.*, copy and move constructors don't count). I looked through 100 uses of the term *converting constructor* on Stack Overflow questions and answers, counting multiple uses within the same question/answer as a single use, and categorized them according to their apparent meaning. In 4 cases the author appeared to be unaware of the fact that explicit constructors do not qualify as converting constructors. The remaining 96 cases broke down as follows:

- Converting constructor can accept any number of argument: 13
- Converting constructor must accept one argument: 21

- Inconclusive: 62
- Converting constructor includes copy/move constructor: 14
- Converting constructor excludes copy/move constructor: 29
- Inconclusive: 53

Indications as to what meaning was intended included, but were not limited to: citing of explicit definitions (such as the one in the Standard or the one on cppreference); explicitly stating that a copy/move constructor or a multi-argument constructor is a converting constructor; explicitly stating that a converting constructor must have a parameter whose type is different from that of the constructor's class; using “copy constructor” and “converting constructor” as disjoint categories; stating that a class has no converting constructor (when in fact it does have a non-explicit copy constructor and in some cases a non-explicit default constructor); stating that a class has one converting constructor (thus excluding the copy constructor); or referring to “the” converting constructor of a class in a context in which such a designation would be ambiguous unless it was implicitly understood to exclude the copy constructor (for example, referring to the constructor of `std::variant` with a single forwarding reference parameter as “the” converting constructor). Inconclusive cases included ones where a one-parameter constructor other than a copy/move constructor was referred to as a converting constructor; a suggestion was made to the question author to add a converting constructor; converting constructors were compared and contrasted with conversion functions generally; or phrasing like “the converting constructor that takes U” was used.

Is this situation a problem?

One possible response is that it doesn't really matter what term the Standard uses, as long as the meaning of the Standard is clear. I disagree with this, because I think that the terms chosen by the Standard clearly have influence on how the C++ community talks about C++ constructs. For example, sometimes C++ programmers use the term “member” (of a class) to refer to any entity that belongs to the class's scope, while sometimes they mean only data members. Having two different definitions for “member” is unfortunate, but the Standard has chosen the former, which in some sense makes the latter “wrong”; the Standard is viewed as the authoritative source of definitions for terms that refer to C++ constructs, so anyone who is using a contradictory definition is expected to align their usage with that of the Standard. Similarly, the Standard's consistent usage of “parameter” and “argument” helps to move the entire C++ community in the direction of using these terms to properly distinguish between the entities declared by the function and the expressions used at the call site.

One might then suggest attempting to educate the C++ community into using the term “converting constructor” with the meaning given to it by the Standard. I think this is unlikely to succeed because the Standard's claim on the definition is weak: unlike “member”, “parameter”, and “argument”, which are each used hundreds of times in the Standard by text prescribing their behavior and their relationship to other C++ constructs, “converting constructor” is used only a few times, so that one rarely needs to apply any of the rules that refer to the term. Such rules “anchor” the term to the Standard's meaning: in order to understand what the rule means, you need to understand the meanings of the terms that the rule applies to. “Converting constructor” is tenuously anchored: every time someone writes those two words, they're vastly more likely to simply be using the term to refer to a category of constructors than to be discussing the properties given to such constructors by the Standard. As a result, they are prone to guessing what it means (and “a non-explicit constructor with one parameter, other than a copy/move constructor” is the intuitively obvious meaning) without coming to a realization that this guess is wrong. The same then happens when others read what has been written. They are unlikely to encounter someone who knows the Standard's definition.

Thus, it appears that as long as the Standard continues to use the current definition of “converting constructor” we will have a situation where two competing definitions exist, and the one in the Standard appears to be the less useful, less obvious one. We do the community a disservice by perpetuating this situation: the Standard tells them that the way they're using the term is wrong, and neither the C++ community, nor even members of this

Committee, benefit much from the Standard's definition. Those people who know both definitions are confused: when they read the words “converting constructor” they must figure out whether the author meant it as the Standard does, or as most other C++ programmers do; similarly, if they want to write the words “converting constructor” with the meaning that is more useful, they must grapple with the fact that anyone who knows the Standard's definition will be confused. Having two different definitions of the term makes the term harder to use.

So what should we do about it?

Remove the definition of “converting constructor” from the Standard and replace all uses of it by “non-explicit constructor”.

It might be argued that deleting the term from the Standard makes all existing references to it (from outside the Standard) dangling. I think this is not really the case, just like how old answers that talk about POD classes are still comprehensible despite the fact that the Standard no longer defines “POD class”. In any case, I think that to the extent that the references become dangling, there are upsides as well: the people who are currently most likely to know the Standard's definition are also the ones who are most likely to be aware that the Standard no longer defines the term, which will spur them to start writing “non-explicit constructor” instead. I find this resulting future to be more desirable than one in which the “official” definition remains in the minority forever.

Wording

Wording is relative to [N4993](#).

Edit [class.conv.ctor]/1:

A constructor that is not explicit (9.2.3) specifies a conversion from the types of its parameters (if any) to the type of its class. ~~Such a constructor is called a converting constructor.~~

Strike [class.conv.ctor]/3:

~~A non-explicit copy/move constructor ([class.copy.ctor]) is a converting constructor.
[Note 2: An implicitly-declared copy/move constructor is not an explicit constructor; it can be called for implicit type conversions. — end note]~~

Edit [over.match.ctor]/1:

[...] Otherwise, the candidate functions are all the ~~converting~~non-explicit constructors (11.4.8.2) of that class. [...]

Edit [over.match.copy]/1:

[...] Assuming that “cv1 T” is the type of the object being initialized, with T a class type, the candidate functions are selected as follows:

- The ~~converting~~non-explicit constructors of T are candidate functions.
- [...]

Edit [over.match.list]/1:

[...] In copy-list-initialization, if an explicit constructor is chosen, the initialization is ill-formed.
[Note 1: This differs from other situations (12.2.2.4, 12.2.2.5), where only ~~converting~~non-explicit constructors are considered for copy-initialization. This restriction only applies if this initialization is part of the final result of overload resolution. — end note]